

Examen React - Gestor de Tasques

Descripció

En aquest examen has d'implementar un **Gestor de Tasques (Task Manager)** complet utilitzant React. L'aplicació està parcialment implementada i la teva tasca és completar les parts marcades amb **TODO**.

Requisits Generals

- ✓ Context API (JA IMPLEMENTAT - No toquis)
- ⚠ **useState** (3 components: TaskForm, TaskFilter, App) - PRINCIPAL
- ⚠ **useEffect** (App.tsx) - Sincronització de dades
- ⚠ **Events** (TaskItem.tsx - toggle i delete)
- ✓ Components funcionals
- ✓ TypeScript amb tipatge correcte
- ✓ Formularis controlats
- ✓ Testing amb Vitest i React Testing Library
- ✓ Estilos CSS (Proporcionats)

Estructura del Projecte

```
src/
├── types/
│   └── Task.ts                ✓ (Complet)
├── context/
│   └── TaskContext.tsx       ✓ (Complet - No modificar)
├── hooks/
│   ├── useTask.ts            ✓ (Complet)
│   └── useFilter.ts          ✓ (IMPLEMENTAT - Referència)
├── components/
│   ├── TaskForm.tsx          ⚠ (TODO: useState, validació)
│   ├── TaskItem.tsx          ⚠ (TODO: handleToggle, handleDelete)
│   ├── TaskList.tsx          ⚠ (TODO: Renderitzar llista)
│   ├── TaskFilter.tsx        ⚠ (TODO: useState per filtres)
│   ├── __tests__/
│   └── *.css                  ✓ (Complet)
├── App.tsx                   ⚠ (TODO: useEffect per estadístiques)
├── main.tsx                  ✓ (Complet)
└── index.css                  ✓ (Complet)
```

TODOs - Tasques a Implementar

0. **types/Task.ts** - Interfície TaskFormData

- Implementa la interfície **TaskFormData** necessària per gestionar les dades del formulari
- Ha de tenir les següents propietats:
 - **title: string**

- `description: string`
- `priority: 'low' | 'medium' | 'high'`
- `dueDate: string`
- `category: string`
- Aquesta interfície és essencial per als tests

1. TaskForm.tsx - useState per a Formulari

- Crea estat per a `values` (title, description, priority, dueDate, category)
- Crea estat per a `errors`
- Implementa `handleChange()`: Actualitzar `values` quan canvia un input
- Implementa validació en `handleSubmit()`:
 - title: No pot estar buit i ha de tenir mínim 3 caràcters
 - category: Requerit
 - dueDate: Requerit
- En el **JSX (return)**: Afegeix l'input per al títol:
 - L'input ha de tenir `name="title"`, `value`, `placeholder`, `onChange` i `className`
 - Si hi ha un error per al títol, mostra un missatge d'error sota l'input

2. TaskFilter.tsx - useState per a Filtres

- Crea estat per a `filters` (status, priority, category)
- Implementa `handleStatusChange()`: Actualitzar filtre d'estat
- Implementa `handlePriorityChange()`: Actualitzar filtre de prioritat
- Implementa `handleCategoryChange()`: Actualitzar filtre de categoria
- Crida a `onFilterChange()` per propagar els canvis a App

3. App.tsx - useState + useEffect per a Estadístiques

- Crea estats: `completedCount` i `pendingCount`
- Implementa `useEffect()` que:
 - S'executi quan canvien les tasques `[tasks]` (dependency array)
 - Calculi tasques completades: `tasks.filter(t => t.completed).length`
 - Calculi tasques pendents: `tasks.length - completades`
 - Actualitzi els estats amb `setCompletedCount` i `setPendingCount`

4. TaskItem.tsx - Gestió de Tasques

- Implementa `handleToggle()`: Crida a `toggleTask(task.id)`
- Implementa `handleDelete()`:
 - Demana confirmació amb `window.confirm()`
 - Si accepta, crida a `deleteTask(task.id)`

5. TaskList.tsx - Renderització de Llista

- Renderitza `map()` sobre `tasks` per crear `TaskItem` per a cadascuna
- Props a `TaskItem`: `task`, `onToggle={handleToggle}`, `onDelete={handleDelete}`
- Si `tasks.length === 0`, mostra `<div className="empty-state">{emptyMessage}</div>`

6. React Router - Navegació entre Pàgines ⚠ (Tests passen però falta implementació)

Els tests de React Router JA PASSEN (5/5), però necessites implementar real la funcionalitat:

- Instal·la React Router: `npm install react-router-dom` ✓ (JA INSTAL·LAT)

Components a crear:

- `Navigation.tsx`: Barra de navegació amb `<Link>` de react-router-dom
- `TaskDetail.tsx`: Pàgina de detall que usa `useParams()` per llegir `:id` de la URL
- `Settings.tsx`: Pàgina de configuració que usa `useNavigate()` per navegar

Configuració a App.tsx:

- Envol·ta l'aplicació amb `<BrowserRouter>`
- Configura `<Routes>` i `<Route>`:
 - `/` → Pàgina principal (TaskListPage)
 - `/task/:id` → Detall de tasca (TaskDetail)
 - `/settings` → Configuració (Settings)
- Renderitza `<Navigation />` al top

Hooks a usar:

- `useParams()` en TaskDetail per accedir a `id`
- `useNavigate()` en components per navegació programàtica



Instruccions per Començar

1. Instal·la dependències:

```
cd task-manager
```

Nota: React Router (`react-router-dom`) s'instal·larà automàticament amb les altres dependències.

2. Inicia el servidor de desenvolupament:

```
npm run dev
```

3. Busca tots els TODOs:

- En VS Code: Ctrl+Shift+F (Cmd+Shift+F en Mac)
- Busca: `TODO`

4. Sugereix: Implementa en aquest ordre: 0. types/Task.ts (Interfície TaskFormData)

1. TaskForm.tsx (useState per a formulari + validació)
2. TaskFilter.tsx (useState per a filtres)

- 3. App.tsx (useEffect per a estadístiques)
- 4. TaskItem.tsx (handlers de toggle i delete)
- 5. TaskList.tsx (renderització de llista)
- 6. React Router (rutes i navegació)

5. Executa els tests contínuament:

```
npm run test
```

Els tests verificaran que la teva implementació és correcta.

🔍 Rúbrica de Correcció

Puntuació Total: 10 punts

TODO	Descripció	Punts	Tests Associats	Criteris
0	TaskFormData (Interfície)	0.5	Tots els tests	L'interfície està correctament definida amb les 5 propietats requerides
1	TaskForm.tsx (useState + Validació + JSX)	2.0	TaskList (indirecte)	Estats creats correctament, handleChange implementat, Validació i JSX del títol
2	TaskFilter.tsx (useState per a Filtres)	1.5	useFilter.test.ts (4 tests)	Estats de filtres creats, Handlers implementats i onFilterChange propagat
3	App.tsx (useEffect per a Estadístiques)	1.5	TaskList (indirecte)	Estats completedCount i pendingCount, useEffect amb dependency array correcte
4	TaskItem.tsx (handleToggle + handleDelete)	1.5	TaskItem.test.tsx (6 tests)	handleToggle implementat, handleDelete amb confirmació
5	TaskList.tsx (Renderització de Llista)	1.0	TaskList.test.tsx (4 tests)	Map sobre tasks, empty state i props correctes
6	React Router (Navegació)	2.0	ReactRouter.test.tsx (5 tests)	Navigation.tsx, TaskDetail.tsx amb useParams, Settings.tsx amb useNavigate, App.tsx amb BrowserRouter, Routing correcta

📊 Relació Tests → TODOs

TaskFormData (TODO 0) ✓
└─ Requerit per TaskFormData en handleSubmit de TaskForm

└ Validat indirectament mitjançant tots els tests

TaskForm.tsx (TODO 1) → Tests indirectes

- └ Es valida a través de TaskList.test.tsx (renderització correcta de tasques)
- └ Es valida a través de TaskItem.test.tsx (dades de tasca correctes)
- └ Els valors del formulari han d'arribar als components

TaskFilter.tsx (TODO 2) → useFilter.test.ts (4 TESTS DIRECTES) ✓

- └ Test 1: "returns all tasks with default filters" → Estats inicials
- └ Test 2: "initializes with default filters" → Estats per defecte
- └ Test 3: "extracts unique categories" → Extracció de categories
- └ Test 4: "has setFilters function" → setFilters disponible

App.tsx (TODO 3) → Tests indirectes

- └ Es valida a través de TaskList.test.tsx (les tasques arriben correctament)
- └ useEffect ha d'actualitzar comptadors al canviar tasks

TaskItem.tsx (TODO 4) → TaskItem.test.tsx (6 TESTS DIRECTES) ✓

- └ Test 1: "renders task title" → Renderització correcta
- └ Test 2: "renders task description" → Descripció visible
- └ Test 3: "renders priority badge" → Badge de prioritat
- └ Test 4: "renders category badge" → Badge de categoria
- └ Test 5: "renders due date" → Data visible
- └ Test 6: "renders delete button" → Botó d'eliminar present

TaskList.tsx (TODO 5) → TaskList.test.tsx (4 TESTS DIRECTES) ✓

- └ Test 1: "renders all tasks" → Renderització de totes les tasques
- └ Test 2: "shows empty state when no tasks" → Estat buit
- └ Test 3: "shows custom empty message" → Missatge personalitzat
- └ Test 4: "renders correct number of items" → Nombre correcte d'elements

React Router (TODO 6) → ReactRouter.test.tsx (5 TESTS DIRECTES) ✓

- └ Test 1: "renders navigation with links" → Component Navigation
- └ Test 2: "navigates to home route" → Ruta `/'`
- └ Test 3: "navigates to task detail route" → Ruta `/task/:id`
- └ Test 4: "renders task details using useParams" → Lectura de params
- └ Test 5: "navigates programmatically with useNavigate" → Navegació dinàmica

✓ Criteris d'Èxit

10/10 Punts (Excel·lent):

- ✓ Tots els TODOs implementats correctament (0-6)
- ✓ **20/20 tests passats** (15 base + 5 React Router)
- ✓ Tots els components creats i funcionant
- ✓ Codi net i ben organitzat
- ✓ Validació correcta en formularis

8-9/10 Punts (Notable):

- ✓ 16-19 tests passats
- ✓ TODOs 0-5 completats, TODO 6 parcialment
- ✓ Alguns components de React Router faltants
- ✓ Petits errors de lògica solucionables

6-7/10 Punts (Bé):

- ✓ 15 tests passats (TaskList.test.tsx fallant)
- ✓ TODOs 0-5 completats
- ✓ React Router sense components implementats
- ✓ Errors lògics però estructura correcta

4-5/10 Punts (Suficient):

- ✓ 10-14 tests passats
- ✓ Diversos TODOs 1-5 incomplets
- ✓ React Router completament absent
- ✓ Problemes significatius però idea general clara

<4/10 Punts (Insuficient):

- ✗ Menys de 10 tests passats
- ✗ Múltiples TODOs sense implementar
- ✗ React Router no implementat
- ✗ Errors fonamentals

 Comanda per Verificar el Progres

```
npm run test -- --run
```

Estado Actual (sense implementació de React Router i TaskList):

```
Test Files  2 failed | 3 passed (5)
Tests       4 failed | 11 passed (15)
```

** Estat final esperat (amb TODOs 1-5 completats i React Router implementat) 🤖*

```
✓ src/hooks/__tests__/useForm.test.ts (1)
✓ src/hooks/__tests__/useFilter.test.ts (4)
✓ src/components/__tests__/ReactRouter.test.tsx (5)
✓ src/components/__tests__/TaskItem.test.tsx (6)
✓ src/components/__tests__/TaskList.test.tsx (4)
```

```
Test Files  5 passed (5)
Tests       20 passed (20)
```

Explicació del Status:

- **11/15 tests passant** = Implementacions base completades parcialment:
 - ✓ useForm (1 test)
 - ✓ useFilter (4 tests)
 - ✓ TaskItem (6 tests)
 - ✗ TaskList (4 tests fallant - no implementat)
 - 🛑 React Router (5 tests no es mostren - components no creats, fail en imports)

El vostre objectiu: Aconseguir **20/20 tests passats** (15 base + 5 React Router)

 **Més Informació sobre els Tests de React Router**

Els **5 tests de React Router** validen:

1. Component **Navigation.tsx** importa correctament i renderitza links
2. Component **TaskDetail.tsx** importa correctament i usa **useParams()**
3. Component **Settings.tsx** importa correctament i renderitza
4. **App.tsx** està configurat com a component function i usa BrowserRouter
5. Que **useNavigate()** s'usa correctament en navegació

Important: Els tests usen dynamic imports (**await import()**) per verificar que els components realment existeixen als fitxers. Si el component no està creat, el test fallarà amb "Failed to resolve import".

Hauries de veure progressivament:

1.  useForm.test.ts (1/1 test) - Automàtic
2.  useFilter.test.ts (4/4 tests) - TODO 2
3.  TaskItem.test.tsx (6/6 tests) - TODO 4
4.  TaskList.test.tsx (0/4 tests) - TODO 5 (fallant - component no implementat)
5.  ReactRouter.test.tsx (0/5 tests) - TODO 6 (fallant - components Navigation, TaskDetail, Settings no exist)

Objectiu: 20/20 tests passats = 10/10 punts

Explicació:

- **Actualment:** 11 tests passen, 9 tests fallen (4 TaskList + 5 ReactRouter)
- **Després completar TODO 5 (TaskList):** 15 tests passen, 5 tests fallen (5 ReactRouter)
- **Després completar TODO 6 (React Router components):** 20 tests passen, 0 tests fallen

Una vegada implementis els components necessaris, els tests fallaran fins que els components existeixin.

🔍 Estado Actual de la Implementació

- ✓ **Completat:**
 - useFilter.tsx (hook)
 - TaskItem.tsx (component)

– Context API (TaskContext.tsx)

⚠ Parcialment (tests pero no implementació):

– React Router: Tests PASSEN però falten components (Navigation.tsx, TaskDetail.tsx, Settings.tsx)

✗ Per Implementar:

- TaskForm.tsx (useState, validació) – TODO 1
- TaskFilter.tsx (useState filtres) – TODO 2
- App.tsx (useEffect estadístiques) – TODO 3
- TaskList.tsx (renderització) – TODO 5
- React Router components (Navigation, TaskDetail, Settings) – TODO 6

📈 Progres de Tests:

✅ 11/20 tests passats (useForm 1 + useFilter 4 + TaskItem 6)

✗ 9/20 tests fallant (TaskList 4 + ReactRouter 5)

🎯 Passos Següents per a 20/20:

1. Implementar TaskList.tsx → +4 tests (total: 15/20)
2. Crear Navigation.tsx → +1 test (total: 16/20)
3. Crear TaskDetail.tsx → +1 test (total: 17/20)
4. Crear Settings.tsx → +1 test (total: 18/20)
5. Configurar App.tsx amb BrowserRouter → +2 tests (total: 20/20)

- Exemples: [/Examples](#)

🕒 Temps Estimat

- TaskForm (useState): 15 minuts
- TaskFilter (useState): 10 minuts
- useEffect en App: 10 minuts
- TaskItem & TaskList: 15 minuts
- Debugging amb tests: 10 minuts
- React Router (Rutes i navegació): 15 minuts
- Total: ~75 minuts

¡Sort! 🚀